

Documento de Diseño Técnico (TDD): Proyecto "Taco'Os" v1.1

1. Misión y Propósito Estratégico

Misión: Democratizar la inteligencia financiera para micro-negocios informales.

Propósito: Transformar la gestión de negocios desde la intuición hacia la toma de decisiones informada, garantizando la salud financiera y evitando el cierre por desinformación.

Filosofía: "Finanzas como el alma del negocio". El sistema debe ser un aliado silencioso: analiza, alerta y sugiere, sin abrumar.

2. El Nicho (El Mercado Abandonado)

Atendemos al sector desatendido por el software empresarial tradicional. Nuestras herramientas actuales son ineficientes y complejas; Taco'Os no crea una nueva categoría, mejora radicalmente la existente mediante:

Interfaz de Usuario "3 Botones": La complejidad técnica reside en el backend; el usuario solo interactúa con tres botones principales de acción inmediata.

Cero Esfuerzo de Aprendizaje: El sistema se adapta al usuario, no el usuario al sistema.

Modelo de Negocio: Volumen mediante bajo costo (estrategia Freemium con escalabilidad Premium).

3. Arquitectura del Sistema (Estructura Modular)

A. Capa de Control Financiero (Backend - Spring Boot)

Módulo Transaccional: Registro de ventas, gastos y deudas (Cuentas por pagar).

Motor de Alerta Temprana: Algoritmo que monitorea el flujo de caja. Si el sistema detecta riesgo de quiebra (incapacidad de pago a proveedores o empleados), dispara una alerta contextual con acciones correctivas.

Integración de Deudas: Gestión de pasivos (ej. "Proveedor de pasteles - \$10,000") para visualización clara de la capacidad de pago.

B. Capa de Inteligencia (Motor IA - Batch Processing)

Analítica Progresiva: La IA solo se activa si el usuario muestra un comportamiento consistente.

Nivelación de Consejos:

Free: Control básico, flujo de caja, 1 insight semanal.

Premium: Consejos diarios, análisis de lealtad, proyecciones de flujo, estrategia de compras.

C. Capa de Fidelización (Estrategia Anti-Fraude)

CRM de WhatsApp: Envío de recibos digitales.

Valor para el usuario: Elimina el fraude del cajero (el cliente exige su comprobante).

Valor para el cliente: Programa de recompensas (ej. "10 tacos = 1 regalo").

Valor para el sistema: Captura de datos masiva para alimentar el motor de IA.

4. Diseño de la Interfaz (UX de "Simplicidad Radical")

La interfaz debe evitar la fatiga mental:

Dashboard Principal: Solo los tres botones de acción rápida.

Visualización: Gráficos simplificados al máximo (evitar métricas complejas, mostrar solo "estás ganando" o "necesitas ajustar X").

Flujos: Guiar al usuario por el camino más corto. Si el sistema detecta que el usuario va a cerrar el mes con pérdidas, la acción "Sugerida" debe ser lo primero que vea al abrir la app.

5. Roadmap Técnico (Fases)

Fase	Enfoque	Entregable Principal
------	---------	----------------------

- | | | |
|-----|-------------------|--|
| I | Control Operativo | App (Flutter) con CRUD de ventas/gastos/deudas y flujo de caja simple. |
| II | Integración de IA | Motor de análisis de datos y sistema de fidelización/recibos (WhatsApp). |
| III | Digitalización QR | Menú interactivo QR, gestión de inventario en tiempo real. |

6. Consideraciones de Futuro (Escalabilidad)

Aunque ahora nos enfocamos en el sector micro, la estructura técnica (Spring Boot + PostgreSQL) permite:

Escalabilidad a Grandes Administraciones: El motor de datos es capaz de migrar a sistemas robustos si el cliente madura.

Integración Bancaria (Potencial): Al tener datos de comportamiento financiero real y transaccional histórico, el sistema está posicionado para, eventualmente, servir como "score" para micro-créditos de aliados financieros.

Siguiente paso sugerido:

Como desarrollador senior, te propongo definir las entidades del "Módulo Transaccional". Necesitamos que las deudas y las ventas convivan en un esquema que permita calcular el cash flow real sin errores.

¿Quieres que definamos la estructura de las tablas transacciones y deudas para asegurar que el sistema pueda alertar cuando alguien esté por quebrar?

Documento de Diseño Técnico (TDD): Proyecto "Taco'Os" v1.1

1. Misión y Propósito Estratégico

Misión: Democratizar la inteligencia financiera para micro-negocios informales. **Propósito:** Transformar la gestión de negocios desde la intuición hacia la toma de decisiones informada, garantizando la salud financiera y evitando el cierre por desinformación. **Filosofía:** "Finanzas como el alma del negocio". El sistema debe ser un aliado silencioso: analiza, alerta y sugiere, sin abrumar.

2. El Nicho (El Mercado Abandonado)

Atendemos al sector desatendido por el software empresarial tradicional. Nuestras herramientas actuales son ineficientes y complejas; **Taco'Os no crea una nueva categoría, mejora radicalmente la existente** mediante:

- **Interfaz de Usuario "3 Botones":** La complejidad técnica reside en el backend; el usuario solo interactúa con tres botones principales de acción inmediata.
- **Cero Esfuerzo de Aprendizaje:** El sistema se adapta al usuario, no el usuario al sistema.
- **Modelo de Negocio:** Volumen mediante bajo costo (estrategia *Freemium* con escalabilidad *Premium*).

3. Arquitectura del Sistema (Estructura Modular)

A. Capa de Control Financiero (Backend - Spring Boot)

- **Módulo Transaccional:** Registro de ventas, gastos y **deudas (Cuentas por pagar)**.
- **Motor de Alerta Temprana:** Algoritmo que monitorea el flujo de caja. Si el sistema detecta riesgo de quiebra (incapacidad de pago a proveedores o empleados), dispara una alerta contextual con acciones correctivas.
- **Integración de Deudas:** Gestión de pasivos (ej. "Proveedor de pasteles - \$10,000") para visualización clara de la capacidad de pago.

B. Capa de Inteligencia (Motor IA - Batch Processing)

- **Análítica Progresiva:** La IA solo se activa si el usuario muestra un comportamiento consistente.
- **Nivelación de Consejos:**
 - *Free:* Control básico, flujo de caja, 1 insight semanal.
 - *Premium:* Consejos diarios, análisis de lealtad, proyecciones de flujo, estrategia de compras.

C. Capa de Fidelización (Estrategia Anti-Fraude)

- **CRM de WhatsApp:** Envío de recibos digitales.
 - *Valor para el usuario:* Elimina el fraude del cajero (el cliente exige su comprobante).
 - *Valor para el cliente:* Programa de recompensas (ej. "10 tacos = 1 regalo").
 - *Valor para el sistema:* Captura de datos masiva para alimentar el motor de IA.

4. Diseño de la Interfaz (UX de "Simplicidad Radical")

La interfaz debe evitar la fatiga mental:

1. **Dashboard Principal:** Solo los tres botones de acción rápida.
2. **Visualización:** Gráficos simplificados al máximo (evitar métricas complejas, mostrar solo "estás ganando" o "necesitas ajustar X").

3. **Flujos:** Guiar al usuario por el camino más corto. Si el sistema detecta que el usuario va a cerrar el mes con pérdidas, la acción "Sugerida" debe ser lo primero que vea al abrir la app.

5. Roadmap Técnico (Fases)

Fase Enfoque	Entregable Principal
I	Control Operativo App (Flutter) con CRUD de ventas/gastos/deudas y flujo de caja simple.
II	Integración de IA Motor de análisis de datos y sistema de fidelización/recibos (WhatsApp).
III	Digitalización QR Menú interactivo QR, gestión de inventario en tiempo real.

6. Consideraciones de Futuro (Escalabilidad)

Aunque ahora nos enfocamos en el sector micro, la estructura técnica (Spring Boot + PostgreSQL) permite:

- **Escalabilidad a Grandes Administraciones:** El motor de datos es capaz de migrar a sistemas robustos si el cliente madura.
- **Integración Bancaria (Potencial):** Al tener datos de comportamiento financiero real y transaccional histórico, el sistema está posicionado para, eventualmente, servir como "score" para micro-créditos de aliados financieros.

Siguiente paso sugerido:

Como desarrollador senior, te propongo **definir las entidades del "Módulo Transaccional"**.

Necesitamos que las **deudas** y las **ventas** convivan en un esquema que permita calcular el *cash flow* real sin errores.

¿Quieres que definamos la estructura de las tablas transacciones y deudas para asegurar que el sistema pueda alertar cuando alguien esté por quebrar?

3.1. Estrategia de Conectividad y Persistencia (Offline-First)

Para garantizar la inmediatez de la experiencia, implementaremos un modelo de **Sincronización Diferida**:

1. **Local-First (SQLite/Room):** La base de datos maestra para la operación del cajero reside en el dispositivo. Cada venta o gasto se registra localmente al instante (latencia 0).
2. **Sincronización en Segundo Plano (Background Sync):**
 - La app intentará sincronizar con el backend en la nube (Spring Boot) cada 5-10 minutos.

- Si el dispositivo detecta pérdida de conexión, el usuario continúa operando normalmente sin interrupciones.

3. Gestión de Conflictos y Re-conexión:

- En cuanto el dispositivo recupere internet, un *Job Scheduler* enviará las transacciones pendientes a la API mediante un *batch update*.
- **Consistencia:** El "dueño" recibirá la actualización del estado real de su negocio en el momento que los dispositivos se sincronicen, permitiendo que la IA analice datos consolidados sin importar que el internet haya fallado temporalmente.

Implementación del "Modelo de los 3 Botones" (UX)

Para que el sistema sea adoptado por quienes "odian las opciones", la interfaz se estructura así:

- **Botón 1: "+" Venta.** (Despliega selección rápida de productos o monto libre).
- **Botón 2: "-" Gasto.** (Registro rápido de salida de dinero/deuda).
- **Botón 3: "¿Cómo voy?".** (Visualización del Flujo de Caja y Alertas de IA).

Nota: La IA procesa los datos en la nube. Si el usuario es "Free", solo verá sus números. Si es "Premium", recibirá el consejo del día en este mismo botón.

1. El Concepto: "La Ventana al Crecimiento"

En lugar de un banner publicitario estático, usa un **"Card de Insight Bloqueado"** dentro de la sección "¿Cómo voy?".

- **Comportamiento:** El usuario entra a "¿Cómo voy?". Ve sus números (ventas, gastos). Debajo de sus números, aparece una tarjeta con un icono de "IA/Insights" que dice: *"Negocios como el tuyo aumentaron sus ventas 15% este mes usando [Estrategia]. ¿Quieres ver cómo? Activar Premium"*.
- **Por qué funciona:** No interrumpes el flujo principal (los 3 botones funcionan perfecto). Solo cuando el usuario *busca* analizar su negocio, le das la opción de ir más allá. Es el momento donde el usuario está más receptivo.

2. Integración en el Flujo de Sincronización

Como acordamos que la sincronización es en segundo plano, esto es técnicamente elegante:

- Si el usuario es **Free**, el backend ignora los cálculos complejos.
- Si el usuario es **Premium**, el sistema (batch job) procesa los datos y el banner muestra un insight real y personalizado.
- Si el usuario es **Free**, el banner siempre muestra un "Insight Genérico de la Comunidad" (como un *preview*). Esto le da valor inmediato sin haber pagado, y le genera curiosidad por el insight *personalizado*.

3. Diseño del Banner ("No molestar")

Para que sea "bonito" y no molesto:

- **Color:** Usa un color neutro o que combine con la app, no el típico rojo de "¡ERROR, PAGA AHORA!".
- **Ubicación:** Siempre al final de la pantalla de "¿Cómo voy?". Nunca al inicio. Que el usuario primero vea *su* trabajo y *sus* números.
- **Micro-copy:** "Desbloquea el análisis de ventas para tu negocio" suena mucho mejor que "Hazte Premium".

Tu TDD actualizado con esta lógica de monetización:

- **Estrategia de Conversión:** El valor se entrega primero (control y claridad), la monetización se presenta como una "mejora de nivel" (IA y consejos avanzados) cuando el usuario ya confía en la app.
- **UX de Monetización:** Banner de sugerencia de upgrade situado exclusivamente en el feed secundario de resultados, con lenguaje aspiracional ("Mejora tu negocio") y nunca de bloqueo ("Debes pagar").

Documento de Diseño Técnico: Ecosistema Taco'Os (v1.2)

1. Misión y Propósito

Misión: Democratizar la inteligencia financiera para micro-negocios.

Propósito: Que el micro-empresario tome el control de su dinero y mejores decisiones mediante datos reales, evitando el cierre por desinformación.

Filosofía: "Finanzas como alma del negocio". Soluciones simples, adecuadas y sin fricción técnica.

2. El Mercado y la Oportunidad

- **Sector:** Micro-negocios informales (el nicho olvidado por el software tradicional).
- **Ventaja Competitiva:** No es un software contable, es una herramienta de crecimiento. Superamos a la "libreta" mediante rapidez y valor agregado.
- **Estrategia de Adopción:** "UX de 3 Botones". Cero curva de aprendizaje.

3. Arquitectura del Sistema (Offline-First)

El sistema opera bajo un modelo de **sincronización diferida** para garantizar que la app nunca sea un obstáculo operativo.

- **Persistencia Local:** Base de datos SQLite/Room en el dispositivo para latencia cero.

- **Sincronización en Segundo Plano:** El dispositivo envía datos al backend (Spring Boot) cada 5-10 minutos o al detectar red estable.
- **Consistencia:** El backend actúa como consolidador, permitiendo al dueño ver el estado real del negocio desde su propia sesión, independientemente de qué dispositivo (cajero) haya registrado la operación.

4. Estructura del Ecosistema (Roadmap)

Fase I: Pilar Operativo

- **Funcionalidad:** Registro de ventas, gastos y deudas. Flujo de caja en tiempo real.
- **UX:** App Flutter con 3 botones principales.
- **Backend:** API robusta en Spring Boot con soporte para *Batch Sync*.

Fase II: Inteligencia y Fidelización

- **IA de Optimización:** Algoritmos de análisis sobre datos consolidados (batch).
- **CRM WhatsApp:** Envío de recibos digitales (anti-fraude) y programa de recompensas (lealtad).
- **Upselling:** Tarjeta no intrusiva de "Insights Premium" en la sección "¿Cómo voy?".

Fase III: Digitalización del Punto de Venta

- **QR Menú:** Menú digital interactivo en mesa.
- **Stock:** Gestión de inventario en tiempo real desde el dispositivo del cajero.

5. Modelo de Datos (Esquema Esencial)

Entidad	Propósito
Business	Configuración, plan suscrito (Free/Premium).
Transaction	Registro (venta, gasto, deuda), estado is_synced, timestamp.
Insight	Mensajes de IA vinculados al negocio (predeterminados o personalizados).
Customer	Captura de lealtad (vía WhatsApp/Promos).

6. UX: Simplicidad Radical

La interfaz se rige por la **ley del mínimo esfuerzo**:

1. **Dashboard Principal:** Solo los 3 botones de acción (Venta, Gasto, ¿Cómo voy?).

2. **Banner Premium:** Ubicado discretamente al final de la pantalla "¿Cómo voy?". Nunca interrumpe el registro ni el flujo operativo.
3. **Feedback:** Notificaciones breves y claras. Nada de jerga contable compleja.

7. Estrategia de Crecimiento y Monetización

- **Free:** 1 Negocio, control operativo básico, acceso a insights genéricos de la comunidad.
- **Premium:** Multi-negocio, IA personalizada, CRM WhatsApp, gestión de deudas.
- **Visión a largo plazo:** Al poseer datos financieros reales y patrones de comportamiento, el sistema constituye un activo de alto valor para futuras integraciones financieras (micro-créditos, fondeo, expansión).

Estado del Proyecto:

- **Estrategia:** Validada y alineada a las necesidades reales del mercado.
- **Prioridad:** Desarrollo del flujo local (Offline) y el endpoint de sincronización masiva (/sync).
- **Siguiente Paso:** Definir el contrato de la API (JSON) para que tu aplicación Flutter sepa exactamente cómo enviar las transacciones locales hacia tu backend Spring Boot de manera eficiente.

Documento de Diseño Técnico: Ecosistema Taco'Os (v1.3)

1. Onboarding: "Entrar y Vender"

- **Registro:** Login social (Google) obligatorio. Es nuestra base de confianza.
- **Roles:** Al abrir la app, el usuario elige:
 - **Dueño:** Crea el negocio, define la suscripción, añade cajeros.
 - **Cajero:** Escanea QR o ingresa número del patrón. Enlace inmediato.
- **Configuración "Just-in-Time":** No hay menús de configuración inicial. Si el cajero va a cobrar y no hay productos, la app le sugiere: "¿Qué vas a cobrar hoy?" y permite crear el producto **ahí mismo** (nombre + precio).

2. Flujo de Venta (UI/UX Radical)

- **Entrada de Venta: * Cliente (Opcional):** Si está vacío, se asigna un folio genérico. Si se ingresa el número, se activa el CRM.
 - **Producto:** Lista de acceso rápido.
 - **Cálculo:** Botón grande de "Cobrar" -> Pop-up simple (Total: \$115, Pago: \$500, Cambio: \$385).

- **Anti-fraude (Cancelaciones):** Permitir cancelar solo dentro de un margen de 3 minutos.

Requiere:

1. Selección de causa (selección rápida).
2. Foto del producto cancelado (evidencia obligatoria).
3. Notificación push inmediata al celular del dueño.

3. Seguridad y Arquitectura de Acceso

- **Tokens:** JWT con caducidad diaria (24h) para forzar la re-validación de sesión y asegurar que el cajero de turno sea quien dice ser.
- **Integridad:** Cada transacción incluye user_id, device_id y timestamp. Si el cajero intenta borrar algo, el log en el backend es inmutable.

4. Evolución del Valor (El "Efecto Juan Ramos")

Tú lo has dicho: **Juan Ramos, dueño de 2 taquerías, es un activo.**

- Al registrar ventas, gastos y deudas, el sistema automáticamente genera un perfil financiero.
- Si el sistema detecta que Juan tiene un flujo constante, la IA no solo le da "consejos", sino que puede empezar a ofrecerle "**soluciones de negocio**" (ej. "Tus ventas suben 20% los viernes, ¿necesitas más insumos? Aquí tienes un proveedor de carne aliado").
- La app principal es el **pilar de datos** que permite estas futuras integraciones de valor.

5. Resumen del Esquema de Datos para el Cajero

Para que esto sea rápido, la app móvil debe manejar localmente:

- **Tabla LocalProducts:** (id, nombre, precio, category_id). Si el cajero crea algo, se guarda aquí y se sincroniza al backend.
- **Tabla LocalTransactions:** (id, customer_phone, total, items_json, status).

Consideraciones técnicas para ti (Backend/Flutter):

1. **JWT Dinámico:** Tu API debe validar el token, pero en la app Flutter, usa SecureStorage para mantener la sesión activa sin que el cajero tenga que loguearse cada vez, solo re-validar el token al inicio del turno.
2. **Sincronización:** Cuando el cajero hace un cambio, envías el payload { "action": "cancel", "reason": "...", "photo": "base64/url" }. El backend recibe esto y, si el dueño tiene activadas notificaciones, disparas un *push notification* al instante.

1. Mapeo del Flujo de Venta (El Camino del Cajero y el Patrón)

Este flujo es el mismo para ambos, garantizando que el usuario no sienta que está usando un "sistema complicado".

1. Inicio de Turno:

- **Cajero:** Login -> "Abrir Caja" (Carga saldo inicial).
- **Patrón:** Login -> (Si el patrón está en el punto de venta, simplemente pulsa "Cobrar" y el sistema detecta que él es el usuario con rol de ADMIN).

2. Registro de Venta (El "Happy Path"):

- **Selección:** Lista rápida de productos. Si no existe, "Agregar al vuelo" (nombre + precio).
- **Cliente:** Campo opcional (si se llena, se activa CRM).
- **Cobro:** Botón "Cobrar" -> Cálculo de cambio.
- **Cierre:** Acción de "Cobrar" dispara (1) Guardado local, (2) Envío a nube en segundo plano, (3) Envío de WhatsApp al cliente (opcional).

3. Cancelación (Vigilancia del Patrón):

- **Acción:** Cajero pulsa "Cancelar".
- **Validación:** Sistema verifica timestamp ($t < 3 \text{ min?}$).
- **Captura:** Cajero selecciona motivo y toma foto (obligatoria).
- **Notificación:** Backend dispara *Push Notification* al celular del Patrón: "Cancelación detectada en [Negocio] - [Cajero]. Ver evidencia."

2. Estructura de Control de Activos (Licenciamiento)

Para gestionar los límites (Free vs Premium), usaremos una tabla de ActiveAssets en tu backend.

Plan	Límites de Activos
FREE	1 Patrón + 2 Cajeros (Máx. 1 Negocio).
PREMIUM	1 Patrón + 5 Cajeros (Máx. 2 Negocios).
BUSINESS	Multi-Patrón + Ilimitados Cajeros (Acceso total).

- **Validación Técnica:** Cada vez que un usuario intenta "Enlazarse" (cajero escaneando QR de patrón), el backend consulta `count(employees) where business_id = X`. Si alcanza el límite, el sistema bloquea el enlace y muestra: "Tu plan actual alcanzó el límite de cajeros. Mejora a Premium para agregar más."

3. Diagrama de Flujo (Lógica de Usuario)

Plaintext

[LOGIN]

|

+--> ¿Es Cajero?

| |--> [Ingresa Tel. Patrón] --> [Enlace Automático] --> [PANTALLA DE VENTA]

|

+--> ¿Es Patrón?

|--> [Dashboard Admin] --> [Gestionar Cajeros/Licencias] --> [Vender]

4. Detalles Técnicos para el TDD (Vigilancia del Patrón)

Para la "**Vigilancia del Patrón**", implementaremos un **Dashboard de Auditoría en Tiempo Real**:

- **Endpoint de Auditoría:** GET /api/v1/business/{id}/audit.
- **Contenido:**
 - Lista de cancelaciones (con foto y motivo).
 - Historial de arqueos de caja (diferencias detectadas).
 - Alertas de "No registro": Si el flujo de caja del día es anormalmente bajo comparado con el promedio histórico del patrón, el sistema envía un *push*: *"Tus ventas están un 30% por debajo del promedio. ¿Todo bien en el punto de venta?"*

1. UX: El "Calendario de Bolsillo"

El patrón no quiere excels complejos. Quiere ver qué pasó hoy, ayer o la semana pasada.

- **Selector de Rango:** Un componente de calendario limpio que permite:
 - **Preset:** Botones rápidos de "Hoy", "Ayer", "Esta Semana", "Este Mes".
 - **Custom:** Selección de fecha inicio y fecha fin para periodos específicos (ej. "del 15 al 20 de mayo").
- **Visualización de Resultados (El "Resumen de un vistazo"):**
 - Total Ventas (Ingresos).
 - Total Gastos (Salidas).
 - **Balance Real (Caja):** (Ingresos - Gastos).
 - *Nota:* Si hay deudas pendientes, mostrar una alerta pequeña debajo del balance: *"Tienes \$X pendientes por pagar a proveedores"*.

2. Estructura Técnica del Reporte (Backend)

Para que esto sea rápido incluso con miles de registros, necesitamos que el backend entregue **datos agregados**, no la lista completa de transacciones.

Endpoint propuesto:

GET /api/v1/business/{id}/reports?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD

- **Respuesta JSON esperada:**

JSON

```
{
  "period": "2026-06-01 to 2026-06-05",
  "summary": {
    "total_sales": 15450.00,
    "total_expenses": 3200.00,
    "net_cash": 12250.00,
    "transaction_count": 84
  },
  "top_products": [
    {"name": "Tacos al Pastor", "quantity": 120},
    {"name": "Coca-Cola", "quantity": 45}
  ]
}
```

3. La "Vigilancia" (Filtro de Auditoría)

Dentro de este mismo módulo, el patrón tendrá una pestaña llamada **"Auditoría"**, donde podrá ver:

1. **Bitácora de Cancelaciones:** Lista de las cancelaciones hechas por los cajeros, con el motivo y la foto de evidencia.
2. **Reporte por Cajero:** ¿Cuánto vendió Juan vs. cuánto vendió María? (Esto ayuda al patrón a identificar quién es más eficiente o dónde hay sospechas de fraude).
3. **Comparativa de Tiempos:** ¿A qué hora se vendió más? (Datos clave para que la IA le diga después: *"Deberías tener 2 cajeros los viernes de 8pm a 10pm"*).

4. Filosofía del Reporte: "Información, no Ruido"

- **Si el balance es positivo:** El color de la tarjeta de reporte es **verde**.

- **Si el balance es negativo (o está en riesgo):** El color es **ámbar/rojo** y el sistema sugiere un botón: *"Ver detalle de gastos"*.
- **Exportación:** Un botón de "Compartir" que genera un PDF ultra-simple o un mensaje de texto formateado para WhatsApp. (A los patrones les encanta mandar sus reportes por WhatsApp a sus socios o familiares).

Roadmap de implementación para este módulo:

1. **Fase 1 (MVP):** Reporte simple de Ingresos vs. Egresos por rango de fechas (el JSON que definimos arriba).
2. **Fase 2:** Agregar el filtro por "Cajero" y la visualización de "Cancelaciones".
3. **Fase 3:** Integración con la IA para que, al ver un reporte, aparezca el botoncito: *"¿Quieres saber por qué vendiste menos el martes? (IA Premium)"*.